

Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

Author: Candidate Engineering Submission **Scope:** 1-2 Pages Research & Product Thesis ---

1. Problem Statement: Why LLD Practice is Hard

Software engineers preparing for senior engineering roles, architecture interviews, or internal promotion loops frequently struggle with Low-Level Design (LLD). Unlike Data Structures & Algorithms (DSA)—where correctness is binary and easily verifiable via automated test inputs and outputs—LLD involves evaluating:

- **Object Responsibility Allocation:** Single Responsibility Principle (SRP) and cohesive class boundaries.
- **Interface & Abstraction Quality:** Polymorphic extensibility, loose coupling (DIP), and design pattern selection.
- **Trade-off Reasoning:** Balancing over-engineering vs. under-engineering under real-world constraints.
- **Edge Case & Concurrency Hardening:** Managing state transitions, locks, idempotency, and component failures.

The Learner Paradox

A learner can spend 2 hours designing a Parking Lot, Elevator System, or Vending Machine on a whiteboard or code editor, but remains uncertain whether their abstraction choices are actually good. Existing preparation tools either provide static reference solutions (leading to passive memorization) or unconstrained AI prompts (leading to generic, non-actionable feedback). ---

2. Research on Existing Approaches & Tools

We evaluated four prevailing paradigms in software engineering practice:

Approach / Tool	Format	Feedback Mechanism	Major Gaps & Limitations
LeetCode / HackerRank	Unit Test Code	Pass/Fail Boolean + Runtime/Memory	Evaluates runtime algorithms, not object-

	Execution	Benchmarks	oriented design, class relationships, or SOLID principles.
Grokking LLD / GitHub Curations	Static Text / Uml Diagrams	Fixed Reference Solutions ("Answer Key")	No feedback on candidate's custom solution. Encourages memorization rather than architectural trade-off thinking.
Pramp / Peer Mock Interviews	Human Peer Conversation	Peer Scoring Rubric (1-5 Stars) + Verbal Feedback	High scheduling latency, inconsistent interviewer quality, and lack of reproducible evidence-based grading.
Generic LLM Chat (ChatGPT/Claude)	Freeform Prompting	General Unstructured Text ("Looks good!")	Unconstrained LLMs frequently grant 100% scores, miss missing interface abstractions, or hallucinate arbitrary advice without reference to a fixed rubric.

3. Key Product Gaps Identified

From our research, three critical product gaps emerged: 1. **Lack of Evidence-Based Feedback:** Learners do not need generic scores; they need feedback tied directly to line citations or class definitions in their submission (e.g., "*Class `ParkingLot` directly instantiates `PaymentProcessor` via `new` instead of receiving `PaymentStrategy` interface*"). 2. **Missing State & Progression History:** Current platforms evaluate submissions in isolation. Learners cannot observe how Attempt #2 improved upon Attempt #1's design flaws. 3. **Over-reliance on Binary Reference Answers:** LLD allows multiple valid solutions. Evaluation must judge adherence to design principles (SRP, OCP, DIP) rather than exact class name matching. ---

4. Product Thesis & Focused Direction

Our solution is a **Focused LLD Practice Studio** built around a tight 6-step feedback loop: $\text{\texttt{Choose Problem}} \rightarrow \text{\texttt{Design Abstractions}} \rightarrow \text{\texttt{Submit}}$

Multi-Facet Solution} \rightarrow \text{Hybrid Evaluation} \rightarrow \text{Review Evidence Feedback} \rightarrow \text{Iterate Attempt \#2} \}

Key Design Pillars of the MVP:

1. **Multi-Facet Submission:** Learner provides Code Implementation, Class-Responsibility Mappings, Design Trade-off Explanations, and optional Diagram/Mermaid representations. 2. **Hybrid Evaluation Engine:** Combines **deterministic static rule-checks** (syntax sanity, signature presence, god-object smells) with **LLM architectural reasoning** constrained by a rigid 6-axis rubric. 3. **Resilient Asynchronous Pipeline:** Submissions are saved immediately into a state machine (SUBMITTED $\rightarrow$ EVALUATING $\rightarrow$ COMPLETED). Evaluation runs in non-blocking background threads with automatic heuristic fallbacks if LLM APIs are offline. 4. **Attempt Progression & Comparison:** Retains attempt history to allow side-by-side v1 vs v2 score and code comparison.

LLD Practice Platform — Assignment Deliverable